



Algoritmo e Programação L04

Lista TAD

Ex01: O que é um Tipo Abstrato de Dados (TAD) e qual a característica fundamental na sua utilização?

Resposta:

Um **Tipo Abstrato de Dados (TAD)** é uma estrutura de dados que é definida pela sua interface e pelas operações que podem ser realizadas sobre ela, sem se preocupar com a implementação interna. A característica fundamental dos TADs é a **abstração**, que permite ao programador usar a estrutura de dados sem precisar conhecer os detalhes de sua implementação.

Características dos TADs

1. **Encapsulamento:** Os dados e as operações que manipulam esses dados são agrupados. Isso permite ocultar os detalhes da implementação e expor apenas a interface necessária para interagir com os dados.
2. **Interface Definida:** O TAD define um conjunto de operações que podem ser realizadas. Por exemplo, um TAD para uma lista pode incluir operações como inserir, remover e acessar elementos.
3. **Independência da Implementação:** A forma como os dados são armazenados e manipulados é separada da forma como eles são usados. Isso significa que é possível mudar a implementação (por exemplo, de uma lista ligada para um vetor) sem afetar o código que utiliza o TAD.
4. **Modularidade:** Os TADs promovem a organização do código em módulos, facilitando a manutenção e a reutilização de código.

Exemplos Comuns de TADs

- **Listas:** Estruturas que permitem armazenar sequências de elementos.
- **Filas:** Estruturas que permitem o armazenamento de elementos em uma ordem FIFO (First In, First Out).
- **Pilas:** Estruturas que armazenam elementos em uma ordem LIFO (Last In, First Out).
- **Árvores:** Estruturas hierárquicas usadas para organizar dados.
- **Grafos:** Estruturas que representam relações entre elementos.

Em resumo, um TAD é uma maneira de organizar e manipular dados de forma que a implementação interna fique oculta e a utilização se torne mais simples e intuitiva.

Ex02: Quais as vantagens de se programar com TADs?

Resposta:

Programar com Tipos Abstratos de Dados (TADs) oferece várias vantagens que podem melhorar a qualidade e a eficiência do desenvolvimento de software. Aqui estão algumas das principais vantagens:

1. Abstração

- Os TADs permitem que os programadores trabalhem com conceitos de alto nível sem se preocupar com os detalhes da implementação. Isso simplifica o processo de desenvolvimento e permite que os desenvolvedores se concentrem na lógica do programa.

2. Encapsulamento

- A encapsulação protege os dados internos de alterações indesejadas. Os detalhes de implementação ficam ocultos, e o acesso aos dados é feito apenas através de funções ou métodos definidos na interface do TAD. Isso ajuda a evitar erros e a garantir a integridade dos dados.

3. Reutilização de Código

- TADs promovem a reutilização, pois podem ser implementados uma vez e usados em diferentes partes de um programa ou em diferentes projetos. Isso reduz a duplicação de código e facilita a manutenção.

4. Facilidade de Manutenção

- Como a implementação está separada da interface, é mais fácil modificar ou melhorar um TAD sem afetar o restante do código que o utiliza. Se um TAD precisar de uma nova funcionalidade ou uma otimização, as mudanças podem ser feitas na implementação sem impactar os usuários do TAD.

5. Modularidade

- TADs ajudam a organizar o código em módulos, tornando-o mais legível e gerenciável. Cada TAD pode ser desenvolvido e testado de forma independente, o que facilita a colaboração em equipes de desenvolvimento.

6. Facilidade de Testes

- Com os TADs, é possível testar cada estrutura de dados de forma isolada antes de integrá-la ao sistema completo. Isso ajuda a identificar e corrigir erros mais facilmente.

7. Flexibilidade

- A mudança na implementação de um TAD não requer alterações no código que o utiliza, desde que a interface permaneça a mesma. Isso dá aos desenvolvedores flexibilidade para otimizar ou substituir estruturas de dados conforme necessário.

8. Clareza e Organização

- O uso de TADs melhora a clareza do código, pois as operações e comportamentos dos dados são bem definidos. Isso facilita a compreensão do código, especialmente para novos desenvolvedores que estão se familiarizando com o sistema.

Resumo

Em suma, programar com TADs promove uma abordagem mais estruturada e organizada, que pode resultar em código mais eficiente, legível e fácil de manter. Essas vantagens tornam os TADs uma prática comum em programação moderna.

Ex03: Quais as vantagens de se programar com TADs?

Resposta:

Abaixo está uma especificação de um sistema de controle de reservas de um clube que aluga quadras poliesportivas, utilizando Tipos Abstratos de Dados (TADs).

Especificação do Sistema de Controle de Reservas de Quadras Poliesportivas

Objetivo

O sistema deve permitir que usuários reservem quadras poliesportivas, verifiquem a disponibilidade das quadras e cancelem reservas. O sistema deve ser capaz de gerenciar informações sobre usuários, quadras e reservas de forma eficiente.

Estruturas de Dados (TADs)

1. TAD Usuário

- **Descrição:** Representa um usuário do sistema.

• Atributos:

- `id`: Identificador único do usuário.
- `nome`: Nome do usuário.
- `email`: Endereço de e-mail do usuário.

• Operações:

- `criarUsuario(nome: string, email: string): Usuario`
- `obterId(usuario: Usuario): int`
- `obterNome(usuario: Usuario): string`
- `obterEmail(usuario: Usuario): string`

2. TAD Quadra

- **Descrição:** Representa uma quadra poliesportiva do clube.

• Atributos:

- `id`: Identificador único da quadra.
- `tipo`: Tipo de quadra (ex: futebol, basquete, vôlei).
- `localizacao`: Localização da quadra dentro do clube.

• Operações:

- `criarQuadra(tipo: string, localizacao: string): Quadra`
- `obterId(quadra: Quadra): int`
- `obterTipo(quadra: Quadra): string`
- `obterLocalizacao(quadra: Quadra): string`

4. TAD SistemaDeReservas

- **Descrição:** Representa o sistema de controle de reservas de quadras.

• Atributos:

- `usuarios`: Lista de usuários registrados.
- `quadras`: Lista de quadras disponíveis.
- `reservas`: Lista de reservas feitas.

• Operações:

- `adicionarUsuario(nome: string, email: string): Usuario`
- `adicionarQuadra(tipo: string, localizacao: string): Quadra`
- `reservarQuadra(usuario: Usuario, quadra: Quadra, dataHora: DateTime, duracao: int): Reserva`
- `cancelarReserva(reserva: Reserva): bool`
- `verificarDisponibilidade(quadra: Quadra, dataHora: DateTime, duracao: int): bool`
- `listarReservasPorUsuario(usuario: Usuario): List<Reserva>`
- `listarReservasPorQuadra(quadra: Quadra): List<Reserva>`

Funcionalidades do Sistema

1. **Cadastro de Usuários:** Permitir que novos usuários se cadastrem no sistema.
2. **Cadastro de Quadras:** Permitir que o administrador adicione quadras disponíveis para reserva.
3. **Reservar Quadra:** Permitir que usuários reservem quadras informando a data, hora e duração.
4. **Cancelar Reserva:** Permitir que usuários cancelem suas reservas.
5. **Verificar Disponibilidade:** Checar se uma quadra está disponível em um determinado horário antes de realizar uma reserva.
6. **Listar Reservas:** Permitir que um usuário veja suas reservas ou que um administrador veja todas as reservas de uma quadra.

Considerações Finais

3. TAD Reserva

- **Descrição:** Representa uma reserva de quadra feita por um usuário.

- **Atributos:**

- `id`: Identificador único da reserva.
- `usuario: Usuario`: Usuário que fez a reserva.
- `quadra: Quadra`: Quadra reservada.
- `dataHora`: Data e hora da reserva.
- `duracao`: Duração da reserva em minutos.

- **Operações:**

- `criarReserva(usuario: Usuario, quadra: Quadra, dataHora: DateTime, duracao: int): Reserva`
- `obterId(reserva: Reserva): int`
- `obterUsuario(reserva: Reserva): Usuario`
- `obterQuadra(reserva: Reserva): Quadra`
- `obterDataHora(reserva: Reserva): DateTime`
- `obterDuracao(reserva: Reserva): int`

Ex04: Sabe-se que um número complexo é escrito da forma $x + iy$, onde $i^2 = -1$, sendo x a sua parte real e y a parte imaginária, ambas representadas por valores reais. Crie um Tipo Abstrato de Dados (TAD) que represente os números complexos com as seguintes funções:

- (a) criar um número complexo;
- (b) destruir um número complexo;
- (c) soma de dois números complexos;
- (d) subtração de dois números complexos;
- (e) multiplicação de dois números complexos;
- (f) divisão de dois números complexos.

Resposta:

```
//main.c
#include <stdio.h>
#include "numero_complexo.h"

int main() {
    NumeroComplexo* a = criarNumeroComplexo(3.0, 4.0);
    NumeroComplexo* b = criarNumeroComplexo(1.0, 2.0);

    printf("Número A: ");
    imprimirNumeroComplexo(a);

    printf("Número B: ");
    imprimirNumeroComplexo(b);

    NumeroComplexo* soma = somarNumerosComplexos(a, b);
    printf("Soma: ");
    imprimirNumeroComplexo(soma);

    NumeroComplexo* subtracao = subtrairNumerosComplexos(a, b);
    printf("Subtração: ");
    imprimirNumeroComplexo(subtracao);

    NumeroComplexo* multiplicacao = multiplicarNumerosComplexos(a, b);
    printf("Multiplicação: ");
    imprimirNumeroComplexo(multiplicacao);

    NumeroComplexo* divisao = dividirNumerosComplexos(a, b);
    if (divisao != NULL) {
        printf("Divisão: ");
        imprimirNumeroComplexo(divisao);
    }

    // Liberar memória
    destruirNumeroComplexo(a);
    destruirNumeroComplexo(b);
    destruirNumeroComplexo(soma);
    destruirNumeroComplexo(subtracao);
    destruirNumeroComplexo(multiplicacao);
    if (divisao != NULL) {
        destruirNumeroComplexo(divisao);
    }

    return 0;
}

//numero_complexo.h
#ifndef NUMERO_COMPLEXO_H
#define NUMERO_COMPLEXO_H

typedef struct {
    double real; // Parte real
    double imaginario; // Parte imaginária
} NumeroComplexo;

// Funções do TAD
NumeroComplexo* criarNumeroComplexo(double x, double y);
void destruirNumeroComplexo(NumeroComplexo* numero);
NumeroComplexo* somarNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b);
NumeroComplexo* subtrairNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b);
NumeroComplexo* multiplicarNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b);
NumeroComplexo* dividirNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b);
void imprimirNumeroComplexo(NumeroComplexo* numero);

#endif // NUMERO_COMPLEXO_H
```

```
//numero_complexo.c
#include <stdio.h>
#include <stdlib.h>
#include "numero_complexo.h"

// Função para criar um número complexo
NumeroComplexo* criarNumeroComplexo(double x, double y) {
    NumeroComplexo* numero = (NumeroComplexo*)malloc(sizeof(NumeroComplexo));
    if (numero != NULL) {
        numero->real = x;
        numero->imaginario = y;
    }
    return numero;
}

// Função para destruir um número complexo
void destruirNumeroComplexo(NumeroComplexo* numero) {
    free(numero);
}

// Função para somar dois números complexos
NumeroComplexo* somarNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b) {
    return criarNumeroComplexo(a->real + b->real, a->imaginario + b->imaginario);
}

// Função para subtrair dois números complexos
NumeroComplexo* subtrairNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b) {
    return criarNumeroComplexo(a->real - b->real, a->imaginario - b->imaginario);
}

// Função para multiplicar dois números complexos
NumeroComplexo* multiplicarNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b) {
    return criarNumeroComplexo(
        a->real * b->real - a->imaginario * b->imaginario,
        a->real * b->imaginario + a->imaginario * b->real
    );
}

// Função para dividir dois números complexos
NumeroComplexo* dividirNumerosComplexos(NumeroComplexo* a, NumeroComplexo* b) {
    double divisor = b->real * b->real + b->imaginario * b->imaginario;
    if (divisor == 0) {
        printf("Erro: Divisão por zero.\n");
        return NULL;
    }
    return criarNumeroComplexo(
        (a->real * b->real + a->imaginario * b->imaginario) / divisor,
        (a->imaginario * b->real - a->real * b->imaginario) / divisor
    );
}

// Função para imprimir um número complexo
void imprimirNumeroComplexo(NumeroComplexo* numero) {
    if (numero->imaginario >= 0)
        printf("%.2f + %.2fi\n", numero->real, numero->imaginario);
    else
        printf("%.2f - %.2fi\n", numero->real, -numero->imaginario);
}
```

Ex05: Crie um Tipo Abstrato de Dados (TAD) que represente o tipo conjunto de inteiros, utilizando uma representação de vetor de inteiros e que contenha as seguintes funções:

- (a) União
- (b) Cria um conjunto vazio
- (c) Insere
- (d) Remove
- (e) Interseção
- (f) Diferença
- (g) Testa se um número pertence ao conjunto
- (h) Menor valor
- (i) Maior valor
- (j) Testa se os conjuntos são iguais

O sistema de controle de reservas de quadras poliesportivas deve ser implementado de maneira a garantir a integridade dos dados, proporcionando um fluxo lógico de operações e facilitando a interação do usuário. A modularidade proporcionada pelos TADs permite que cada componente do sistema seja desenvolvido e testado de forma independente, aumentando a eficiência do desenvolvimento.

(k) Tamanho
(l) Testa se o conjunto é vazio

Resposta:

```
//main.c
#include <stdio.h>
#include "conjunto.h"

int main() {
    // Criação de conjuntos
    Conjunto* conjuntoA = cria_conjunto_vazio(5);
    Conjunto* conjuntoB = cria_conjunto_vazio(5);

    // Inserindo elementos no conjunto A
    insere(conjuntoA, 1);
    insere(conjuntoA, 2);
    insere(conjuntoA, 3);

    // Inserindo elementos no conjunto B
    insere(conjuntoB, 2);
    insere(conjuntoB, 3);
    insere(conjuntoB, 4);

    // Exibindo tamanhos
    printf("Tamanho do Conjunto A: %d\n", tamanho(conjuntoA));
    printf("Tamanho do Conjunto B: %d\n", tamanho(conjuntoB));

    // União
    Conjunto* uniaoAB = uniao(conjuntoA, conjuntoB);
    printf("Tamanho da união: %d\n", tamanho(uniaoAB));

    // Interseção
    Conjunto* intersecaoAB = intersecao(conjuntoA, conjuntoB);
    printf("Tamanho da interseção: %d\n", tamanho(intersecaoAB));

    // Diferença
    Conjunto* diferencaAB = diferenca(conjuntoA, conjuntoB);
    printf("Tamanho da diferença (A - B): %d\n", tamanho(diferencaAB));

    // Testando a pertença
    printf("O elemento 2 pertence ao conjunto A? %s\n", pertence(conjuntoA, 2) ? "Sim" : "Não");

    // Menor e maior valor
    printf("Menor valor do conjunto A: %d\n", menor_valor(conjuntoA));
    printf("Maior valor do conjunto A: %d\n", maior_valor(conjuntoA));

    // Verificando se conjuntos são iguais
    printf("Os conjuntos A e B são iguais? %s\n", sao_iguais(conjuntoA, conjuntoB) ? "Sim" : "Não");

    // Limpando memória
    destroi_conjunto(conjuntoA);
    destroi_conjunto(conjuntoB);
    destroi_conjunto(uniaoAB);
}
```

```
//conjunto.h
#ifndef CONJUNTO_H
#define CONJUNTO_H

typedef struct {
    int *elementos; // Vetor de elementos inteiros
    int tamanho;    // Tamanho atual do conjunto
    int capacidade; // Capacidade do vetor
} Conjunto;

// Funções para manipulação do conjunto
Conjunto* cria_conjunto_vazio(int capacidade);
void destroi_conjunto(Conjunto* conjunto);
int insere(Conjunto* conjunto, int elemento);
int remove_elemento(Conjunto* conjunto, int elemento);
Conjunto* uniao(const Conjunto* a, const Conjunto* b);
Conjunto* intersecao(const Conjunto* a, const Conjunto* b);
Conjunto* diferenca(const Conjunto* a, const Conjunto* b);
int pertence(const Conjunto* conjunto, int elemento);
int menor_valor(const Conjunto* conjunto);
int maior_valor(const Conjunto* conjunto);
int sao_iguais(const Conjunto* a, const Conjunto* b);
int tamanho(const Conjunto* conjunto);
int conjunto_vazio(const Conjunto* conjunto);

#endif // CONJUNTO_H
```

```
//conjunto.c
#include <stdio.h>
#include <stdlib.h>
#include "conjunto.h"

// Cria um conjunto vazio com capacidade inicial
Conjunto* cria_conjunto_vazio(int capacidade) {
    Conjunto* conjunto = (Conjunto*)malloc(sizeof(Conjunto));
    conjunto->elementos = (int*)malloc(capacidade * sizeof(int));
    conjunto->tamanho = 0;
    conjunto->capacidade = capacidade;
    return conjunto;
}

// Destroi um conjunto e libera a memória
void destroi_conjunto(Conjunto* conjunto) {
    free(conjunto->elementos);
    free(conjunto);
}

// Insere um elemento no conjunto
int insere(Conjunto* conjunto, int elemento) {
    if (pertence(conjunto, elemento)) {
        return 0; // Elemento já existe
    }
    if (conjunto->tamanho == conjunto->capacidade) {
        conjunto->capacidade *= 2;
        conjunto->elementos = (int*)realloc(conjunto->elementos, conjunto->capacidade * sizeof(int));
    }
    conjunto->elementos[conjunto->tamanho++] = elemento;
    return 1; // Inserção bem-sucedida
}
```

```

// Remove um elemento do conjunto
int remove_elemento(Conjunto* conjunto, int elemento) {
    for (int i = 0; i < conjunto->tamanho; i++) {
        if (conjunto->elementos[i] == elemento) {
            conjunto->elementos[i] = conjunto->elementos[--conjunto->tamanho]; // Move o último para a posição do removido
            return 1; // Remoção bem-sucedida
        }
    }
    return 0; // Elemento não encontrado
}

// Realiza a união de dois conjuntos
Conjunto* uniao(const Conjunto* a, const Conjunto* b) {
    Conjunto* resultado = cria_conjunto_vazio(a->tamanho + b->tamanho);
    for (int i = 0; i < a->tamanho; i++) {
        insere(resultado, a->elementos[i]);
    }
    for (int i = 0; i < b->tamanho; i++) {
        insere(resultado, b->elementos[i]);
    }
    return resultado;
}

// Realiza a interseção de dois conjuntos
Conjunto* intersecao(const Conjunto* a, const Conjunto* b) {
    Conjunto* resultado = cria_conjunto_vazio(a->tamanho < b->tamanho ? a->tamanho : b->tamanho);
    for (int i = 0; i < a->tamanho; i++) {
        if (pertence(b, a->elementos[i])) {
            insere(resultado, a->elementos[i]);
        }
    }
    return resultado;
}

// Realiza a diferença entre dois conjuntos
Conjunto* diferenca(const Conjunto* a, const Conjunto* b) {
    Conjunto* resultado = cria_conjunto_vazio(a->tamanho);
    for (int i = 0; i < a->tamanho; i++) {
        if (!pertence(b, a->elementos[i])) {
            insere(resultado, a->elementos[i]);
        }
    }
    return resultado;
}

// Testa se um número pertence ao conjunto
int pertence(const Conjunto* conjunto, int elemento) {
    for (int i = 0; i < conjunto->tamanho; i++) {
        if (conjunto->elementos[i] == elemento) {
            return 1; // Pertence ao conjunto
        }
    }
    return 0; // Não pertence
}

// Retorna o menor valor do conjunto
int menor_valor(const Conjunto* conjunto) {
    if (conjunto_vazio(conjunto)) return -1; // Conjunto vazio
    int menor = conjunto->elementos[0];
    for (int i = 1; i < conjunto->tamanho; i++) {
        if (conjunto->elementos[i] < menor) {
            menor = conjunto->elementos[i];
        }
    }
    return menor;
}

// Retorna o maior valor do conjunto
int maior_valor(const Conjunto* conjunto) {
    if (conjunto_vazio(conjunto)) return -1; // Conjunto vazio
    int maior = conjunto->elementos[0];
    for (int i = 1; i < conjunto->tamanho; i++) {
        if (conjunto->elementos[i] > maior) {
            maior = conjunto->elementos[i];
        }
    }
    return maior;
}

// Testa se dois conjuntos são iguais
int sao_iguais(const Conjunto* a, const Conjunto* b) {
    if (a->tamanho != b->tamanho) return 0; // Tamanhos diferentes
    for (int i = 0; i < a->tamanho; i++) {
        if (!pertence(b, a->elementos[i])) {
            return 0; // Elemento não encontrado em b
        }
    }
    return 1; // São iguais
}

// Retorna o tamanho do conjunto
int tamanho(const Conjunto* conjunto) {
    return conjunto->tamanho;
}

// Testa se o conjunto é vazio
int conjunto_vazio(const Conjunto* conjunto) {
    return conjunto->tamanho == 0;
}

```

Ex06: Crie um Tipo Abstrato de Dados (TAD) que represente os números racionais e que contenha as seguintes funções:

- (a) Cria um número racional;
- (b) Soma de números racionais;
- (c) Multiplica números racionais;
- (d) Testa se são iguais.

Resposta:

```

//main.c
#include <stdio.h>
#include "racional.h"

```

```

int main() {

```

```

//racional.c
#include <stdio.h>
#include "racional.h"

```

```

// Função para calcular o máximo divisor comum

```

```
Racional r1 = cria_racional(3, 4);
Racional r2 = cria_racional(2, 5);

Racional soma = soma_racionais(r1, r2);
Racional produto = multiplica_racionais(r1, r2);

printf("Soma: %d/%d\n", soma.numerador, soma.denominador);
printf("Produto: %d/%d\n", produto.numerador, produto.denominador);

if (sao_iguais(r1, r2)) {
    printf("Os números racionais são iguais.\n");
} else {
    printf("Os números racionais são diferentes.\n");
}

return 0;
}

//racional.h
#ifndef RACIONAL_H
#define RACIONAL_H

typedef struct {
    int numerador;
    int denominador;
} Racional;

// Função para criar um número racional
Racional cria_racional(int numerador, int denominador);

// Função para somar dois números racionais
Racional soma_racionais(Racional r1, Racional r2);

// Função para multiplicar dois números racionais
Racional multiplica_racionais(Racional r1, Racional r2);

// Função para testar se dois números racionais são iguais
int sao_iguais(Racional r1, Racional r2);

#endif // RACIONAL_H
```

```
int mdc(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Função para criar um número racional
Racional cria_racional(int numerador, int denominador) {
    Racional r;
    if (denominador == 0) {
        printf("Erro: Denominador não pode ser zero.\n");
        r.numerador = 0;
        r.denominador = 1; // Definindo um padrão
    } else {
        int divisor = mdc(numerador, denominador);
        r.numerador = numerador / divisor;
        r.denominador = denominador / divisor;
    }
    return r;
}

// Função para somar dois números racionais
Racional soma_racionais(Racional r1, Racional r2) {
    Racional resultado;
    resultado.numerador = r1.numerador * r2.denominador + r2.numerador * r1.denominador;
    resultado.denominador = r1.denominador * r2.denominador;

    int divisor = mdc(resultado.numerador, resultado.denominador);
    resultado.numerador /= divisor;
    resultado.denominador /= divisor;

    return resultado;
}

// Função para multiplicar dois números racionais
Racional multiplica_racionais(Racional r1, Racional r2) {
    Racional resultado;
    resultado.numerador = r1.numerador * r2.numerador;
    resultado.denominador = r1.denominador * r2.denominador;

    int divisor = mdc(resultado.numerador, resultado.denominador);
    resultado.numerador /= divisor;
    resultado.denominador /= divisor;

    return resultado;
}

// Função para testar se dois números racionais são iguais
int sao_iguais(Racional r1, Racional r2) {
    return (r1.numerador == r2.numerador && r1.denominador == r2.denominador);
}
```

Ex07: Desenvolva um TAD para um cubo. Inclua as funções de inicializações necessárias e as operações que retornem os tamanhos de cada lado, a sua área e o seu volume.

Resposta:

```
//main.c
#include <stdio.h>
#include "cubo.h"

int main() {
    double lado;

    printf("Digite o tamanho do lado do cubo: ");
    scanf("%lf", &lado);

    // Cria o cubo
    Cubo* cubo = criaCubo(lado);

    // Exibe informações sobre o cubo
    printf("Lado do cubo: %.2lf\n", getLado(cubo));
    printf("Área do cubo: %.2lf\n", calculaArea(cubo));
    printf("Volume do cubo: %.2lf\n", calculaVolume(cubo));

    // Destrói o cubo
    destroiCubo(cubo);

    return 0;
}

//cubo.h
#ifndef CUBO_H
#define CUBO_H

typedef struct {
    double lado; // Tamanho do lado do cubo
} Cubo;

// Funções para manipulação do cubo
Cubo* criaCubo(double lado);
void destroiCubo(Cubo* cubo);
double getLado(Cubo* cubo);
double calculaArea(Cubo* cubo);
double calculaVolume(Cubo* cubo);

#endif // CUBO_H
```

```
//cubo.c
#include <stdio.h>
#include <stdlib.h>
#include "cubo.h"

// Cria um cubo com o lado especificado
Cubo* criaCubo(double lado) {
    Cubo* novoCubo = (Cubo*)malloc(sizeof(Cubo));
    if (novoCubo != NULL) {
        novoCubo->lado = lado;
    }
    return novoCubo;
}

// Destrói o cubo
void destroiCubo(Cubo* cubo) {
    free(cubo);
}

// Retorna o tamanho do lado do cubo
double getLado(Cubo* cubo) {
    return cubo->lado;
}

// Calcula a área total do cubo
double calculaArea(Cubo* cubo) {
    return 6 * (cubo->lado * cubo->lado); // 6 * lado²
}

// Calcula o volume do cubo
double calculaVolume(Cubo* cubo) {
    return cubo->lado * cubo->lado * cubo->lado; // lado³
}
```

Ex08: Desenvolva um TAD para um cilindro. Inclua as funções de inicializações necessárias e as operações que retornem sua altura e raio, a sua área e o seu volume.

Resposta:

```
//main.c
#include <stdio.h>
#include "cilindro.h"

int main() {

//cilindro.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "cilindro.h"
```

```

double raio, altura;

printf("Digite o raio da base do cilindro: ");
scanf("%lf", &raio);
printf("Digite a altura do cilindro: ");
scanf("%lf", &altura);

// Cria o cilindro
Cilindro* cilindro = criaCilindro(raio, altura);

// Exibe informações sobre o cilindro
printf("Raio da base do cilindro: %.2lf\n", getRaio(cilindro));
printf("Altura do cilindro: %.2lf\n", getAltura(cilindro));
printf("Área total do cilindro: %.2lf\n", calculaArea(cilindro));
printf("Volume do cilindro: %.2lf\n", calculaVolume(cilindro));

// Destrói o cilindro
destroiCilindro(cilindro);

return 0;
}

```

```

//cilindro.h
#ifndef CILINDRO_H
#define CILINDRO_H

typedef struct {
    double raio; // Raio da base do cilindro
    double altura; // Altura do cilindro
} Cilindro;

// Funções para manipulação do cilindro
Cilindro* criaCilindro(double raio, double altura);
void destroiCilindro(Cilindro* cilindro);
double getRaio(Cilindro* cilindro);
double getAltura(Cilindro* cilindro);
double calculaArea(Cilindro* cilindro);
double calculaVolume(Cilindro* cilindro);

#endif // CILINDRO_H

```

```

// Cria um cilindro com o raio e altura especificados
Cilindro* criaCilindro(double raio, double altura) {
    Cilindro* novoCilindro = (Cilindro*)malloc(sizeof(Cilindro));
    if (novoCilindro != NULL) {
        novoCilindro->raio = raio;
        novoCilindro->altura = altura;
    }
    return novoCilindro;
}

// Destrói o cilindro
void destroiCilindro(Cilindro* cilindro) {
    free(cilindro);
}

// Retorna o raio da base do cilindro
double getRaio(Cilindro* cilindro) {
    return cilindro->raio;
}

// Retorna a altura do cilindro
double getAltura(Cilindro* cilindro) {
    return cilindro->altura;
}

// Calcula a área total do cilindro
double calculaArea(Cilindro* cilindro) {
    return 2 * M_PI * cilindro->raio * (cilindro->raio + cilindro->altura); //  $2\pi r(r + h)$ 
}

// Calcula o volume do cilindro
double calculaVolume(Cilindro* cilindro) {
    return M_PI * cilindro->raio * cilindro->raio * cilindro->altura; //  $\pi r^2 h$ 
}

```

Ex09: Desenvolva um TAD para uma esfera. Inclua as funções de inicializações necessárias e as operações que retornem seu raio, a sua área e o seu volume.

Resposta:

```

//main.c
#include <stdio.h>
#include "esfera.h"

int main() {
    double raio;

    printf("Digite o raio da esfera: ");
    scanf("%lf", &raio);

    // Cria a esfera
    Esfera* esfera = criaEsfera(raio);

    // Exibe informações sobre a esfera
    printf("Raio da esfera: %.2lf\n", getRaio(esfera));
    printf("Área da superfície da esfera: %.2lf\n", calculaArea(esfera));
    printf("Volume da esfera: %.2lf\n", calculaVolume(esfera));

    // Destrói a esfera
    destroiEsfera(esfera);

    return 0;
}

```

```

//esfera.h
#ifndef ESFERA_H
#define ESFERA_H

typedef struct {
    double raio; // Raio da esfera
} Esfera;

// Funções para manipulação da esfera
Esfera* criaEsfera(double raio);
void destroiEsfera(Esfera* esfera);
double getRaio(Esfera* esfera);
double calculaArea(Esfera* esfera);
double calculaVolume(Esfera* esfera);

#endif // ESFERA_H

```

```

//esfera.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "esfera.h"

// Cria uma esfera com o raio especificado
Esfera* criaEsfera(double raio) {
    Esfera* novaEsfera = (Esfera*)malloc(sizeof(Esfera));
    if (novaEsfera != NULL) {
        novaEsfera->raio = raio;
    }
    return novaEsfera;
}

// Destrói a esfera
void destroiEsfera(Esfera* esfera) {
    free(esfera);
}

// Retorna o raio da esfera
double getRaio(Esfera* esfera) {
    return esfera->raio;
}

// Calcula a área da superfície da esfera
double calculaArea(Esfera* esfera) {
    return 4 * M_PI * pow(esfera->raio, 2); //  $4\pi r^2$ 
}

// Calcula o volume da esfera
double calculaVolume(Esfera* esfera) {
    return (4.0 / 3.0) * M_PI * pow(esfera->raio, 3); //  $(4/3)\pi r^3$ 
}

```